SIMATS ENGINEERING

CO 4 - AT 3 - CLOUD COMPUTING

INDUSTRY PROBLEM SOLVING TASK - SET 1

COURSE CODE	**CSA0511**
NAME	**CHINTHA ABHILASH REDDY**

SET 1 - ANSWERS

1. Big Data Platform - Reverse Engineering of Web, Mobile and Transaction Analytics

A platform processing billions of clicks, mobile events and transactions would normally use a layered distributed architecture. The probable design is as follows.

Distributed storage system: HDFS or cloud object storage such as Amazon S3/Azure Data Lake is likely used for low-cost, fault-tolerant storage. Structured transactional data can also be stored in distributed SQL/NoSQL systems such as Hive, HBase or Cassandra.

Data ingestion: Apache Kafka is a likely event backbone for website clicks and mobile interactions. Apache NiFi, Flume or cloud ingestion services can collect logs and files from different applications. Change Data Capture can continuously copy database transactions into the analytics platform.

Batch processing: Apache Spark is the most probable batch engine because it supports distributed transformations, SQL and machine learning. Hadoop MapReduce may be retained for older large-scale batch jobs. Hive can expose warehouse tables through SQL.

Separation and indexing: Structured data such as customer and transaction tables is stored using schemas and columnar formats such as Parquet or ORC. Semi-structured JSON clickstream and mobile logs are stored in separate raw zones, parsed later and indexed by fields such as customer ID, event type, device, timestamp and region. Elasticsearch/OpenSearch may provide fast searchable indexes.

Real-time analytics: Spark Structured Streaming, Apache Flink or Kafka Streams can consume Kafka topics, calculate live metrics and detect behavioral patterns with seconds-level latency.

Scalability and partitioning: Data is horizontally partitioned by date/time, region, customer key or event type. Kafka topics use many partitions and consumer groups. Storage systems replicate blocks/partitions across nodes. Compute clusters use autoscaling and data-local parallel execution so capacity can increase as traffic grows.

Dashboards and BI: Processed data is written to a warehouse/lakehouse or serving layer. Distributed SQL engines such as Trino/Presto or Spark SQL query the data. Power BI, Tableau or similar BI tools connect through SQL/JDBC/ODBC to display sales, customer, traffic and operational KPIs.

Conclusion: Therefore, a Kafka + distributed storage + Spark/Flink + SQL analytics architecture provides scalable ingestion, reliable processing and both batch and real-time reporting.

2. E-Commerce Real-Time Personalization and Recommendation Platform

The platform must combine browsing behavior and reliable order information quickly enough to personalize the customer experience while supporting regional scale.

Event streaming: Cart additions, product views, checkout events and payments are published as events to Apache Kafka or a managed equivalent. Separate topics can be created for views, carts, orders and payments.

Customer profiling databases: Customer profiles and rapidly changing session features can be stored in Cassandra, DynamoDB, HBase or another scalable NoSQL database. A data lake keeps complete historical clickstream and transaction data.

Synchronization: Each event carries identifiers such as customer/session ID, product ID, timestamp and region. Stream processors join recent session events with profile features and transaction updates. CDC tools synchronize order and payment databases with the streaming layer.

Caching and low latency: Redis is a probable cache for sessions, product features, recommendation results and frequently accessed profile data. This avoids repeated high-latency database queries.

Distributed querying: Trino/Presto, Spark SQL or a cloud warehouse can query order tables together with data-lake clickstream records for analysis.

Machine learning workflow: Historical data is cleaned and transformed into features. Collaborative filtering, content-based recommendation, learning-to-rank or hybrid models are trained offline. Models are validated, registered and deployed as online inference services. Live behavior updates features and re-ranks recommendations.

Merging data: Structured orders are stored in relational/warehouse tables, while browsing JSON logs are transformed into a common schema. Customer ID, session ID, product ID and timestamps act as join keys. A curated lakehouse layer creates unified customer and product views.

Scalability and fault tolerance: Kafka partitioning, replicated brokers, distributed NoSQL storage, checkpointing in stream processors and autoscaled stateless services allow the system to survive node failures and regional traffic spikes.

Dashboards and KPIs: Aggregated events feed analytical tables for conversion rate, cart abandonment, average order value, payment success, recommendation CTR and revenue. BI dashboards query these tables and can refresh in near real time.

Conclusion: This design provides synchronized customer intelligence, low-latency recommendations and reliable analytics at e-commerce scale.

3. Big Data Healthcare System - Patient, Image and Wearable Analytics

Healthcare data is highly heterogeneous, so the probable solution combines relational, object, NoSQL and streaming technologies while enforcing strong governance.

Hybrid database architecture: Structured EHR, prescriptions and billing data are stored in relational databases or a healthcare warehouse. Diagnostic images such as DICOM files are stored in secure object storage/PACS. Wearable sensor streams can use time-series or NoSQL databases. A governed data lake/lakehouse integrates these sources for analytics.

Integration and interoperability: Apache NiFi, Kafka, ETL tools and healthcare integration engines can move data between hospitals, laboratories and insurers. HL7 and FHIR APIs standardize clinical data exchange, while DICOM is used for medical imaging.

Streaming and ML: Kafka collects wearable events; Spark Structured Streaming or Flink calculates real-time health indicators and alerts. Spark MLlib, TensorFlow, PyTorch or scikit-learn can train prediction models for disease risk, readmission or abnormal conditions.

Privacy and encryption: Sensitive data is encrypted in transit with TLS and at rest using strong encryption. Role-based or attribute-based access control limits records to authorized users. Identifiers are masked, tokenized or de-identified for research datasets.

Compliance: Consent, retention policies, data lineage and immutable audit logs record who accessed or changed information. Access follows least privilege and applicable healthcare/privacy regulations.

Clinical analytics pipeline: Source data is validated, standardized and linked to a patient identity. Batch jobs create longitudinal patient histories, while streaming jobs generate current observations. Feature engineering produces clinical variables; validated models calculate risk scores. Results are delivered to clinical dashboards with human review rather than replacing clinician judgment.

Population health: De-identified datasets can be processed with Spark across large populations to study disease prevalence, treatment outcomes, resource utilization and regional trends.

Conclusion: The hybrid architecture enables real-time monitoring and large-scale clinical analytics while protecting sensitive patient information.

4. Smart City Big Data Platform - Traffic, CCTV, Weather and Transport

A smart city platform needs distributed collection close to devices, resilient messaging, geospatial processing and centralized long-term analytics.

Edge computing: Gateways near traffic sensors and cameras perform filtering, protocol conversion, compression and basic inference. Only useful CCTV metadata or required video segments are forwarded, reducing bandwidth and response time.

Messaging: MQTT can connect lightweight IoT devices to gateways, while Apache Kafka is a likely central distributed event bus for traffic, weather and transport streams.

Centralized storage: A cloud/on-premise data lake stores raw and historical data. Object storage holds large files, while curated Parquet tables support analytics. Replication protects against hardware failure.

Geospatial and streaming analytics: Flink or Spark Structured Streaming performs windowed vehicle counts, average speed calculations and congestion prediction. PostGIS, Elasticsearch geo features or distributed spatial frameworks can execute location-based queries.

Time-series and location databases: TimescaleDB, InfluxDB or Cassandra can store high-frequency sensor measurements. PostgreSQL/PostGIS is suitable for road networks, stops, zones and geospatial relationships.

Lambda/lakehouse style analytics: The streaming layer provides live congestion and incident information. Batch Spark jobs analyze months or years of historical data for road design, route planning, public transport scheduling and infrastructure investment. Curated outputs from both layers are combined in an analytical store.

Security: Device certificates, TLS, network segmentation, encryption at rest, RBAC/ABAC and API gateways restrict access. CCTV and citizen-related identifiers should be minimized or anonymized, with complete audit logging.

Visualization: Operational dashboards use map layers, charts, alerts and live KPIs. GIS tools, Grafana, Power BI or Tableau can connect to the serving databases to show congestion, bus positions, weather conditions and infrastructure status.

Conclusion: The probable architecture therefore combines edge intelligence, Kafka-based streaming, geospatial databases and a central lakehouse to support both immediate operations and long-term urban planning.

5. Healthcare Analytics System - Integrated Medical Data and Predictive Analytics

This system is similar to a modern healthcare lakehouse in which secure clinical records, device measurements and unstructured reports are integrated without forcing every source into one database.

Hybrid storage: Relational databases store patients, appointments and structured clinical fields. Object storage keeps reports, documents and medical files. A time-series/NoSQL store manages wearable readings. A governed data lake/lakehouse keeps historical copies and curated analytical tables.

ETL and interoperability: ETL/ELT pipelines using NiFi, Spark or cloud data integration tools validate, clean, deduplicate and standardize records. HL7/FHIR interfaces exchange clinical information between hospitals and clinics; common patient and encounter identifiers support integration.

Real-time monitoring: Wearable events are published to Kafka or an IoT ingestion service. Flink/Spark Structured Streaming applies thresholds, windows and predictive models to identify unusual readings and generate alerts.

Distributed population analysis: Spark distributes joins, aggregations and feature engineering across a cluster. It can analyze millions of records for disease trends, readmission rates, treatment outcomes and risk factors more efficiently than a single server.

Predictive health analytics: Models such as logistic regression, random forest, gradient boosting and neural networks can estimate disease or readmission risk. Recommendation models may rank treatment options using clinical features and approved medical rules. Predictions should include confidence/interpretability and remain subject to clinical validation.

Encryption and governance: TLS protects data in transit and encryption keys protect data at rest. RBAC, MFA, least privilege, consent controls and data masking restrict access. Audit logs record reads, updates and model use; lineage records the source and transformation history.

Compliance and quality: Retention policies, de-identification for secondary analytics, controlled research environments, data-quality rules and periodic access reviews help satisfy healthcare privacy and governance requirements.

Decision support: Validated risk scores and trends are written to a serving database and displayed in clinical dashboards. Alerts can highlight high-risk patients, while population dashboards help administrators plan preventive programs and resources.

Conclusion: The solution combines interoperable ETL, distributed processing, real-time streams and governed machine learning to produce useful healthcare insights while maintaining security and compliance.